

JSON 序列化与反序列化系统设计报告

题目：1.24 可序列化对象系统 – JSON 序列化/反序列化

开发者 1：付文聪 2025012005（学号） fwc25@mails.tsinghua.edu.cn（邮箱） 15546336196（手机）

开发者 2：司汶蓬 2025011999（学号） siwenpeng20070806@outlook.com（邮箱） 15822627080（手机）

1. 研究内容与结论

1.1 研究内容

实现一个符合 JSON 子集（对象、数组、字符串、数字、布尔、null）的序列化与反序列化系统。

采用面向对象设计：JsonValue 抽象基类，派生 JsonObject、JsonArray、JsonString、JsonNumber、JsonBool、JsonNull。

序列化：将内存中的 JSON 树输出为紧凑 JSON 字符串，支持转义（\", \\, \n, \r, \t）。

反序列化：递归下降解析器，将 JSON 文本解析为 JsonValue 树，提供详细的错误位置与信息。

内存管理：容器类在析构时自动释放子节点；parse 返回 unique_ptr 确保所有权转移。

1.2 结论

系统正确实现了 JSON 子集的序列化与反序列化，通过了 20 个正向测试、12 个负例测试及混合嵌套测试，解析错误定位准确；通过析构函数与 unique_ptr 管理内存，未见泄漏风险。满足题目全部要求。

2. 代码验证思路、核心代码与运行结果

2.1 验证思路

设计三类测试：

序列化测试：直接构造各类 JsonValue 对象，比对 serialize() 输出与预期字符串。

正向反序列化测试：提供合法 JSON → 解析 → 再序列化，检查输入与输出是否一致（Round-Trip）。

负例测试：输入非法 JSON，验证解析器安全拒绝并报告合理错误位置。

所有测试用例集成在 main.cpp 中，自动统计通过率。

2.2 核心代码

2.2.1 类层次结构（json_value.h）

```
class JsonValue
{
public:
```

```

        virtual string serialize() const = 0;
        virtual ~JsonValue() {}
};
class JsonObject : public JsonValue { ... };
class JsonArray : public JsonValue { ... };
class JsonString : public JsonValue { ... };
class JsonNumber : public JsonValue { ... };
class JsonBool : public JsonValue { ... };
class JsonNull : public JsonValue { ... };

```

2.2.2 序列化示例 (json_serialize.cpp)

JsonString::serialize() 转义实现:

```

string JsonString::serialize() const
{
    string result = "";
    for (char c : value)
    {
        if (c == '"')
            result += "\\\"";
        else if (c == '\\')
            result += "\\\"";
        else if (c == '\n')
            result += "\\n";
        else if (c == '\r')
            result += "\\r";
        else if (c == '\t')
            result += "\\t";
        else
            result += c;
    }
    result += "\"";
    return result;
}

```

2.2.3 解析器核心 (json_parse.cpp)

递归下降入口:

```

JsonValue* parseValue()
{
    skipWhitespace();
    char ch = peek();
    if (ch == '{')
        return parseObject();
    if (ch == '[')
        return parseArray();
}

```

```

    if (ch == '"')
        return parseStringValue();
    if (ch == 'n')
        return parseNull();
    if (ch == 't' || ch == 'f')
        return parseBool();
    if (ch == '-' || isdigit(ch))
        return parseNumber();
    return fail("unexpected character");
}

```

2.3 编译与运行结果

编译环境：采用 Windows 11 64 位操作系统在 Visual Studio 2026 直接编译运行 Serializable Object System 项目

运行结果摘要（完整输出见附录）：

序列化测试：20 个用例全部匹配预期。

正向反序列化测试：20/20 通过，Round-Trip 一致。

负例测试：12/12 通过（空输入、尾逗号、缺逗号、未闭合字符串、非法转义、截断字面量、多余字符、缺引号等）。

典型案例：

输入 `{"address":{"city":"北京","code":100000}}` 解析后再序列化，输出完全一致，验证嵌套结构与转义正确。

2.4 验证结论

系统正确、健壮地完成了 JSON 子集的序列化与反序列化，错误处理完善，内存安全。代码符合 OOP 设计，可维护性良好。

3. 可能的应用及应用场景

本系统实现的是 JSON 文本与内存对象树之间的双向转换，这一能力在软件工程中具有广泛用途，典型场景包括：

1. 配置文件管理。应用程序将用户设置、界面布局等以 JSON 文件保存，启动时反序列化加载，退出时序列化写回磁盘。常见于游戏、桌面工具及嵌入式设备的轻量配置方案。
2. 网络数据交换。Web 服务与客户端之间常以 JSON 作为请求/响应格式。本系统的 `serialize/parse` 流程对应「对象 → 传输文本 → 对象」的基本链路。
3. 日志与调试输出。将程序内部状态构造成 `JsonObject/JsonArray` 后序列化为可读字符串，便于开发阶段打印结构化信息，比纯文本日志更易于后续分析。

4. 优缺点

4.1 优点

面向对象结构清晰。JsonValue 基类统一接口，六种类型各司其职，新增类型时只需派生并实现 `serialize()`，符合开闭原则。

无外部依赖。仅使用 C++ 标准库，便于在 Visual Studio 中直接编译提交，适合课程作业与离线环境。

测试覆盖较完整。包含序列化、正向解析、Round-Trip 与多种负例，能验证嵌套结构与转义互逆等关键性质。

4.2 缺点

功能范围有限。仅支持 JSON 子集，不支持 `\uXXXX` 转义、科学计数法、注释等扩展特性，键名序列化时也未做转义处理。

内存管理偏手工。容器内部使用裸指针，依赖析构函数释放子节点；若使用不当（重复释放、循环引用）易产生未定义行为。

4.3 可替代的解决方案

若采用成熟 JSON 库，可直接获得完整标准支持、更好的性能与社区维护。

若改进内存与类型设计，如以 `std::unique_ptr<JsonValue>` 贯穿容器内部、或以 `std::variant` 替代继承层次，可减少裸指针带来的风险；对象键可用 `vector<pair>` 保序。

附录：

源代码文件清单（已随报告提交）

json_value.h / json_value.cpp

json_serialize.cpp

json_parse.cpp

main.cpp

运行结果截图

```

===== 序列化测试 =====
1. null: null (期望: null)
2. true: true (期望: true)
3. false: false (期望: false)
4. 整数: 42 (期望: 42)
5. 负数: -10 (期望: -10)
6. 小数: 3.14159 (期望: 3.14159)
7. 空字符串: "" (期望: "")
8. 普通字符串: "hello" (期望: "hello")
9. 带引号: "he said \"hi\"" (期望: "he said \"hi\"")
10. 空数组: [] (期望: [])
11. 单元素数组: [1] (期望: [1])
12. 多元素数组: [1,2,3] (期望: [1,2,3])
13. 混合数组: [1,"hello",true,null] (期望: [1,"hello",true,null])
14. 空对象: {} (期望: {})
15. 单键值对: {"k":"v"} (期望: {"k":"v"})
16. 多键值对: {"a":1,"b":2} (期望: {"a":1,"b":2})
17. 嵌套数组: [[1,2],[3,4]] (期望: [[1,2],[3,4]])
18. 数组嵌套对象: [{"name":"张三"}, {"name":"李四"}] (期望: [{"name":"张三"}, {"name":"李四"}])
19. 对象嵌套对象: {"address":{"city":"北京", "code":100000}} (期望: {"address":{"city":"北京", "code":100000}})
20. 对象嵌套数组: {"name":"张三", "scores":[90,95,85]} (期望: {"name":"张三", "scores":[90,95,85]})

===== 反序列化正向测试(对应序列化 20 用例)=====
[PASS] 1. null
[PASS] 2. true
[PASS] 3. false
[PASS] 4. 整数
[PASS] 5. 负数
[PASS] 6. 小数
[PASS] 7. 空字符串
[PASS] 8. 普通字符串
[PASS] 9. 带转义引号
[PASS] 10. 空数组
[PASS] 11. 单元素数组
[PASS] 12. 多元素数组
[PASS] 13. 混合数组
[PASS] 14. 空对象
[PASS] 15. 单键值对
[PASS] 16. 多键值对
[PASS] 17. 嵌套数组
[PASS] 18. 数组嵌套对象
[PASS] 19. 对象嵌套对象
[PASS] 20. 对象嵌套数组
正向测试: 20/20 通过

===== Round-trip 测试(内存树 -> JSON -> 解析 -> JSON)=====
原始 JSON: {"active":true,"name":"测试","scores":[90,85]}
往返 JSON: {"active":true,"name":"测试","scores":[90,85]}
[PASS] round-trip 一致

===== 负例测试 (非法 JSON 应安全拒绝) =====
[PASS] 空输入 - 正确拒绝, pos=0 msg=empty input
[PASS] 对象尾逗号 - 正确拒绝, pos=7 msg=trailing comma in object
[PASS] 数组尾逗号 - 正确拒绝, pos=3 msg=trailing comma in array
[PASS] 对象缺逗号 - 正确拒绝, pos=7 msg=expected ',' or '}' in object
[PASS] 数组缺逗号 - 正确拒绝, pos=3 msg=expected ',' or ']' in array
[PASS] 字符串未闭合 - 正确拒绝, pos=9 msg=unterminated string
[PASS] 非法转义 \q - 正确拒绝, pos=6 msg=invalid escape sequence '\q'
[PASS] 截断的 true - 正确拒绝, pos=0 msg=expected 'true' or 'false'
[PASS] 截断的 null - 正确拒绝, pos=0 msg=expected 'null'
[PASS] 根值后多余字符 - 正确拒绝, pos=4 msg=trailing characters after JSON value
[PASS] 对象键未加引号 - 正确拒绝, pos=1 msg=expected '"' to start string
[PASS] 缺少值 - 正确拒绝, pos=5 msg=unexpected character '}'
负例测试: 12/12 通过

```

声明：本人保证本报告及所附代码为团队合作完成，分工明确，未抄袭他人成果。

日期：2026 年 6 月 12 日